

[EA&B] TSSMRBench: Benchmarking Memory Systems for Retrieving Correct Temporal Semantic States from Evolving Memories

Hai Lin*

Shenzhen International Graduate School, Tsinghua
University
Shenzhen, China
Pengcheng Laboratory
Shenzhen, China
ngyygm@outlook.com

Hai-Tao Zheng[†]

Shenzhen International Graduate School, Tsinghua
University
Shenzhen, China
Pengcheng Laboratory
Shenzhen, China
zheng.haitao@sz.tsinghua.edu.cn

Jianhua Sun*

Shenzhen International Graduate School, Tsinghua
University
Shenzhen, China
sunjianhua1225@gmail.com

Hui Wang

Pengcheng Laboratory
Shenzhen, China
wangh06@pcl.ac.cn

Abstract

LLM-based agents increasingly rely on external memory systems to maintain information across long-term interactions. However, real-world memories are often not static facts: user preferences, plans, task progress, and event descriptions may evolve through multiple semantic states. Existing benchmarks have advanced the evaluation of long-term memory, temporal reasoning, and knowledge update, but they rarely isolate whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory. To address this gap, we introduce **TSSMRBench**, a benchmark for temporal semantic state memory retrieval.

TSSMRBench formalizes this problem as retrieval over evolving state trajectories and asks whether an agent’s memory system can recover the state evidence required by a query rather than merely a relevant or recent memory. We construct a benchmark with 300 scenarios, 9,000 memory nodes, and 900 multiple-choice questions organized into three task families: *single-state lookup*, *cross-version comparison*, and *temporal ordering*. The tasks differ in how many supporting states they require, enabling evaluation of both retrieval precision and support completeness under shared-pool interference.

Experiments with BM25, FAISS, Mem0, and Graphiti show that current retrieval baselines and memory systems perform relatively well on single-state queries but degrade substantially on cross-version comparison and temporal-ordering queries. Dense retrieval remains the strongest simple baseline, while memory systems expose trade-offs between granularity, support recovery, and latency. These findings show that temporal semantic state retrieval remains challenging for current agent memory systems and suggest that future designs should preserve version history, retrieve coherent state-level evidence, and support stage-aware selection among semantically similar memories.

CCS Concepts

- **Information systems** → **Information retrieval**; *Data mining*;
- **Computing methodologies** → *Natural language processing*.

Keywords

agent memory systems, temporal semantic state retrieval, benchmark evaluation, evolving memories

ACM Reference Format:

Hai Lin, Jianhua Sun, Hai-Tao Zheng, and Hui Wang. 2027. [EA&B] TSSMRBench: Benchmarking Memory Systems for Retrieving Correct Temporal Semantic States from Evolving Memories. In *28th International Conference on Extending Database Technology (EDBT ’27)*, April 6–9, 2027, Lille, France. ACM, New York, NY, USA, 12 pages.

1 Introduction

Large language model agents increasingly rely on external memory systems to sustain behavior across long-running interactions. Survey work on LLM-based agent systems describes the rapid expansion of agent architectures and their growing role in complex problem solving [6]. A recent survey focused specifically on memory further argues that memory is what enables an agent to persist, organize, and selectively recall information across interactions [3]. These systems are expected to remember user preferences, track evolving plans, preserve event histories, and support multi-step decision making over time. As agents move from short exchanges to persistent assistance, memory is becoming a first-class systems component rather than a simple extension of context length.

What makes this setting difficult is that many memories evolve. A plan may be revised, a policy may be amended, and an event description may go through several intermediate stages before reaching its latest form. In such settings, the correct answer is often not the most recent memory, but the memory that was valid at a particular stage. Consider a legal assistant asked which revision

*Hai Lin and Jianhua Sun contributed equally to this work.

[†]Corresponding author.

of a procedural rule first introduced delayed-hearing handling, a project assistant asked to compare the current plan with an earlier draft because an old proposal is being reconsidered, or a financial agent asked to reconstruct the order of several market-state updates before issuing a decision recommendation. These queries require the memory system to recover the right *temporal semantic state*, not merely a topically relevant memory.

This requirement is not well captured by existing evaluation practice. Long-term memory benchmarks such as LoCoMo have substantially improved the evaluation of persistent memory [13]. Temporal and time-sensitive benchmarks such as TimeQA have sharpened our understanding of reasoning under changing information [1]. Retrieval-oriented evaluation has also advanced through RAG-style settings, for which survey work now provides a broad overview of evaluation practices and failure modes [5]. However, these benchmarks usually emphasize retention, relevance, or recency, rather than whether a memory system can distinguish among several semantically similar states of the same evolving object and retrieve the one that matches the queried stage. We discuss these neighboring benchmark families in Section 2. Here, the key point is simple: storing an evolving memory trajectory is not the same as retrieving the stage-correct state from that trajectory.

To study this problem, we introduce **TSSMRBench**, a benchmark for *temporal semantic state memory retrieval* in agent memory systems. TSSMRBench models each scenario as an evolving memory trajectory composed of temporal semantic states and evaluates whether the stage-correct state can be retrieved from a memory system given that trajectory. The benchmark contains three task families: *single-state lookup*, which targets one stage-specific state; *cross-version comparison*, which requires jointly retrieving two stage-specific states for contrast; and *temporal ordering*, which requires recovering several states and arranging them along the correct temporal progression. Figure 1 shows one representative benchmark instance.

The current release of TSSMRBench contains 300 scenarios, 9,000 memory nodes, and 900 multiple-choice questions. We evaluate representative sparse retrieval, dense retrieval, and memory-system baselines under a unified global mixed-pool setting, where all memories are stored together before retrieval and each question must be answered from retrieved evidence. The evaluation target is the memory system used by the agent rather than the underlying answer LLM: answer generation is held fixed so that differences primarily reflect memory retrieval quality. Our baseline design spans lexical and dense retrieval together with recent memory-system architectures such as Mem0 [2] and Graphiti [17]. The results show that current memory systems are far more reliable on single-state questions than on comparison and ordering questions; that many failures arise from retrieving temporally incompatible but semantically adjacent states; and that more complex memory systems do not automatically outperform simpler retrievers once state fidelity, memory granularity, and latency are taken into account.

Our contributions are threefold:

- (1) We introduce TSSMRBench and formalize *temporal semantic state memory retrieval*: the ability to retrieve the stage-correct state from a memory system given an evolving memory trajectory.

Table 1: TSSMRBench at a glance. A scenario is one repository-level release-evolution window, each memory node is one temporal semantic state, and each scenario contributes one question for each task family.

Item	Value
Scenarios	300 repository-level release windows
Memory nodes	9,000 temporal semantic states
Questions	900 multiple-choice questions
Task families	Single-state, cross-version, temporal ordering

- (2) We build a benchmark dataset with 300 scenarios, 9,000 memory nodes, and 900 questions, and organize it into three task families: single-state lookup, cross-version comparison, and temporal ordering.
- (3) We conduct a systematic experimental study showing that current memory systems still struggle on this problem, especially when semantically similar states coexist, and we use these findings to motivate future designs that preserve version history and support stage-aware retrieval.

Outline. Section 2 reviews related work. Section 3 presents the task definition, design principles, and benchmark construction. Section 4 reports the experimental setup and results. Section 5 concludes. An unnumbered Artifacts section appears immediately before the references.

2 Related Work

2.1 Agent Memory Systems

MemGPT frames long-term memory as a memory-management problem and studies how an LLM can move information across different memory regions to support persistent interaction [15]. Letta extends this line of work into a broader framework for agent memory operating across sessions and tasks [15]. Mem0 focuses on practical long-term memory for production systems, emphasizing memory extraction, storage, and retrieval as reusable infrastructure for personalized agents [2]. Graphiti, developed in the Zep ecosystem, organizes memory through a graph-centered representation intended to preserve richer structure over time [17]. A-MEM studies more agentic memory behaviors, including how memory can be selectively maintained and used during ongoing interaction [22].

These memory systems differ substantially in retrieval granularity, memory organization, and update behavior. However, they are typically evaluated through downstream task success, user-level memory recall, or general question answering. Such evaluations are valuable, but they do not isolate a narrower question that is central to our benchmark: when several semantically similar states of the same evolving object coexist in memory, can the memory system retrieve the one that matches the queried stage?

2.2 Long-Term Memory Benchmarks

LoCoMo evaluates conversational memory over long multi-session interactions [13]. LongMemEval studies whether systems can recall and use information distributed across long histories [21]. MemoryAgentBench examines how memory-equipped agents perform

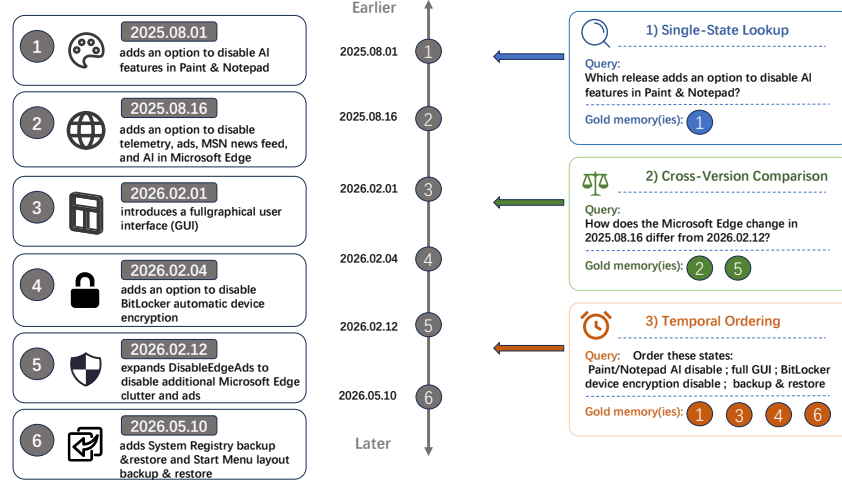


Figure 1: Illustrative benchmark instance from the Raphire/Win11Debloat repository. The full scenario contains 30 temporal semantic states, but only the task-relevant states are shown here. The single-state lookup question depends on one state, the cross-version comparison question depends on two states, and the temporal ordering question depends on four states. The benchmark therefore tests whether a memory system can retrieve the stage-correct supporting states from an evolving memory trajectory, not merely retrieve a topically related memory.

when information from prior interaction needs to be reused in later tasks [7].

Taken together, these benchmarks have moved evaluation beyond single-turn QA and made persistent memory a serious empirical target. At the same time, they mainly treat memory as facts, dialogue details, or episodes to be recalled later. They do not usually build explicit versioned state trajectories for the same underlying object, and they rarely ask for an earlier but stage-correct state when a later state is also present. TSSMRBench complements this line of work by making that distinction explicit.

2.3 Temporal Reasoning and Knowledge Update Benchmarks

Temporal databases and temporal query languages provided early formal tools for representing and querying time-dependent data [8, 18]. In NLP, TORQUE focuses on event-centric temporal reasoning [14]; TimeDial studies temporal commonsense in dialogue [16]; TimeQA targets time-sensitive question answering over evolving facts [1]; ETR-QA provides a comprehensive evaluation of event temporal reasoning abilities [12]; MusTQ introduces multi-step temporal reasoning over temporal knowledge graphs [24]; and Test of Time evaluates LLM temporal reasoning through both semantic understanding and time-arithmetic tasks [4, 20]. These benchmarks are important because they test whether models can reason about order, duration, and time-conditioned facts.

Another neighboring family comes from retrieval and evidence-based QA. HotpotQA emphasizes multi-hop evidence aggregation [23]. NarrativeQA studies understanding over long narrative evidence [9]. Natural Questions evaluates open-domain retrieval and answer extraction from large corpora [10]. MuSiQue focuses on compositional multi-hop questions that require integrating several

pieces of retrieved evidence [19]. RAG and its later variants study how retrieved evidence can be incorporated into generation [5, 11]. These works sharpen our understanding of retrieval and evidence use, but they do not directly target stage-specific state disambiguation inside an evolving memory trajectory.

Knowledge-update settings raise a different but related issue: how a memory system should incorporate newer information and avoid outdated answers. This perspective is highly relevant, but its default assumption is often that newer information ought to dominate older information. TSSMRBench differs in an important way. Our goal is not simply to test whether a memory system prefers the latest state. Instead, we test whether the memory system can retrieve the *correct* state for the queried stage, even when that state is not the most recent one.

3 TSSMRBench

Figure 2 summarizes TSSMRBench. This section defines the task (Section 3.1), presents the benchmark evaluation design principles (Section 3.2), describes the data construction flow (Section 3.3) and question design (Section 3.4), and then introduces the evaluation pipeline and metrics (Section 3.5). TSSMRBench focuses specifically on whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory.

3.1 Task Definition

TSSMRBench evaluates *temporal semantic state memory retrieval*: whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory, rather than merely returning a relevant or recent memory.

Formally, let

$$\mathcal{T} = (m_1, m_2, \dots, m_n)$$

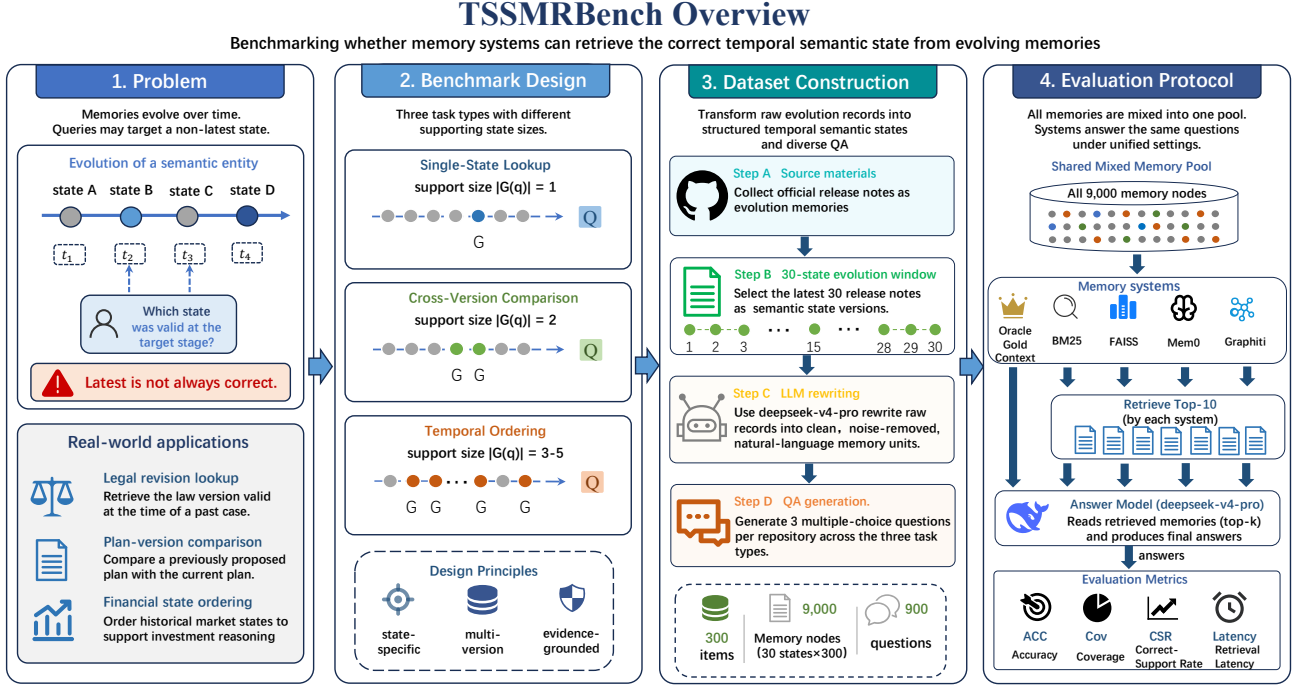


Figure 2: Overall framework of TSSMRBench. The figure summarizes the temporal semantic state retrieval problem, the benchmark task design, the release-derived data construction pipeline, and the unified evaluation protocol. It is intended as a paper-level overview of how the benchmark connects the problem definition, dataset construction, and memory-system evaluation.

Table 2: Comparison of TSSMRBench with representative neighboring benchmarks. LongMem indicates whether the benchmark targets long-term memory; Temporal indicates whether temporal reasoning is a core requirement; Stage-correct retrieval indicates whether the benchmark explicitly asks for the state that is correct for a queried stage rather than merely a relevant or recent memory; Language reports the primary benchmark language; Scale summarizes benchmark size at a coarse level.

Benchmark	Long Mem		Stage-correct retrieval		Lang.	Scale
	Mem	Temporal				
LoCoMo [13]	Yes	Partial	No	EN	1,986 QA	
LongMemEval [21]	Yes	Partial	No	EN	500 QA	
MemoryAgentBench [7]	Yes	Partial	No	EN	Multi-source suite	
TimeQA [1]	No	Yes	No	EN	≈40K QA	
MusTQ [24]	No	Yes	No	EN	≈666K QA	
TSSMRBench	Yes	Yes	Yes	EN	300 repositories / 900 QA	

denote an evolving memory trajectory, where each memory node m_i records a semantic state of the same underlying object, event, entity, or plan, and where the ordering function

$$\tau(m_i) < \tau(m_j)$$

indicates that m_i precedes m_j in the evolution process. A natural-language query q specifies, either explicitly or implicitly, a stage constraint $\sigma(q)$ over this trajectory. Let

$$G(q) \subseteq \mathcal{T}$$

be the minimal set of supporting state nodes sufficient to answer q under $\sigma(q)$, and let

$$R_k(q) = (r_1, r_2, \dots, r_k)$$

be the ranked retrieval output returned by a memory system.

The task is to retrieve a ranked list $R_k(q)$ such that the supporting set $G(q)$ is recovered and the answer generator can derive the gold answer $y(q)$ using only the retrieved evidence. The core difficulty is that several states in $\mathcal{T}$ may be semantically similar while being temporally incompatible with the query.

TSSMRBench operationalizes this capability through three task families:

- **Single-state lookup:** $|G(q)| = 1$. The memory system must identify the one state node that supports the answer.
- **Cross-version comparison:** $|G(q)| = 2$. The memory system must recover the two states required to contrast what changed.
- **Temporal ordering:** $|G(q)| \in \{3, 4, 5\}$. The memory system must recover enough states to reconstruct a local evolution order.

Table 3: Task families and evaluation budgets in TSSMR-Bench. $|G(q)|$ denotes the number of supporting states required by a question, and the listed retrieval top-k values are the answer-time cutoffs reused from one shared top-10 retrieval result.

Task family	$ G(q) $	Retrieval top-k
Single-state lookup	1	1, 2, 3
Cross-version comparison	2	2, 5, 8
Temporal ordering	3–5	5, 8, 10

These tasks differ in support size, but they test the same underlying ability: retrieving the stage-correct state or state set required by the query.

3.2 Benchmark Evaluation Design Principles

To isolate this capability, the benchmark and its evaluation protocol follow five principles. Questions must depend on *stage-specific validity*: the correct answer should be determined by which temporal semantic state is valid for the queried stage, not simply by topical relevance or recency. They must also remain *evidence-grounded*: the final answer should only be derivable by aligning the question’s state description with retrieved memory content, rather than by exploiting superficial cues such as version labels, dates, or option formatting. The benchmark further varies *support requirements* so that some questions depend on one state while others require several states, allowing us to distinguish pointwise retrieval from complete support recovery over a trajectory. The evaluation preserves *mixed-memory interference* by retrieving from repository-mixed memory collections instead of isolated per-trajectory stores. Finally, the protocol separates *retrieval failure* from *state-selection failure*, allowing us to tell whether a memory system missed the required state entirely or instead retrieved semantically related but temporally incompatible evidence.

3.3 Data Construction Flow

We instantiate these principles using an auditable release-only benchmark built from official repository evolution records.

Step 1: Data source selection. We use official GitHub release notes because they are repository-authoritative descriptions of evolving software states, including feature additions, policy changes, fixes, migrations, and deprecations. Compared with informal discussions, release notes provide a cleaner and more reproducible record of how a system changes across versions.

Step 2: Trajectory node determination. For each repository, we take the latest 30 usable release notes and treat them as a fixed local evolution window. Each release note becomes one temporal semantic state node, producing an evolving memory trajectory of length 30. Repositories without a sufficient release history are excluded. The formal benchmark contains 300 repositories, yielding 9,000 memory nodes in total.

Step 3: LLM rewriting of raw release notes. Raw release notes contain hyperlinks, formatting markers, references, and other artifacts that are not ideal as a memory surface. We therefore use a

large language model to rewrite each release note into a normalized English memory unit. This rewriting preserves repository identity and release-specific state content while removing markup-heavy noise. The resulting normalized text is the only memory surface ingested by the evaluated memory systems; the original release note is retained only for audit and provenance.

Step 4: QA generation. On top of each 30-node evolving memory trajectory, we generate one multiple-choice question for each task family, yielding 3 questions per repository and 900 questions overall. For every question, we annotate the supporting state nodes required for answering it. This makes it possible to evaluate not only final answer correctness but also support recovery.

3.4 Question Design

Question design is driven by support complexity rather than by superficial wording diversity.

Single-state lookup. These questions ask the memory system to identify one state node that contains a target condition, decision, fix, or release-specific behavior. They test whether the memory system can localize a single correct state among semantically nearby alternatives.

Cross-version comparison. These questions require contrasting two states that differ in a clear, evidence-supported way. We only construct such questions when the source evidence contains a substantive difference between the two releases. Comparison questions are not generated from pairs whose behavior is essentially unchanged.

Temporal ordering. These questions require mapping several state descriptions back to their supporting nodes and ordering them correctly along the trajectory. They are designed so that solving them robustly still depends on retrieving the corresponding memory content, rather than simply relying on superficial ordering cues in the question text alone.

All questions are multiple-choice. Version identifiers, tags, or publication cues may appear when they are part of the original material, but the question and options are constrained so that the answer cannot be derived reliably without aligning the state descriptions in the query to the stored memory units.

3.5 Evaluation Pipeline and Metrics

All baselines are evaluated under a shared global memory pool: before any query is issued, all 9,000 memory nodes from all 300 repositories are written into each baseline’s own memory backend or retrieval index. Retrieval therefore operates over a single repository-mixed memory collection per baseline, rather than over isolated per-trajectory memory stores. This setting introduces cross-repository interference and prevents the memory system from exploiting trajectory boundaries.

For each query q , the evaluated memory system retrieves the top-10 ranked memories:

$$R_{10}(q) = (r_1, \dots, r_{10}).$$

The same ranked list is then reused across task-specific context budgets:

- **single-state lookup:** $k \in \{1, 2, 3\}$
- **cross-version comparison:** $k \in \{2, 5, 8\}$
- **temporal ordering:** $k \in \{5, 8, 10\}$

The answer LLM is allowed to read only these retrieved memories and is not given access to unretrieved memory nodes. This protocol separates retrieval quality from how much of the ranked memory list is exposed at answer time. It also allows us to analyze whether a memory system places the required support near the top of the ranking or only recovers it deeper in the retrieved list.

We report four primary metrics. **Accuracy (ACC)** measures whether the final answer is correct. **Coverage (Cov)** measures how much of the required support set is recovered:

$$\text{Cov}(q, k) = \frac{|G(q) \cap R_k(q)|}{|G(q)|}.$$

Complete Support Rate (CSR) measures whether the full support set is recovered:

$$\text{CSR}(q, k) = 1[G(q) \subseteq R_k(q)].$$

Latency measures retrieval time only, excluding answer generation.

Because TSSMRBench is intended to diagnose memory retrieval rather than only final QA success, these metrics are designed to jointly capture answer correctness, support completeness, and retrieval cost.

4 Experiments

4.1 Experimental Setup

The formal benchmark is built from 300 repositories, yielding 300 repository-level release-evolution scenarios, 9,000 memory nodes, and 900 multiple-choice questions. We compare five baselines under the same benchmark inputs: BM25, FAISS, Mem0, Graphiti, and Oracle Gold Context.

BM25 indexes the normalized memory units directly and serves as a sparse lexical retriever. FAISS indexes dense node embeddings and serves as a standard semantic retriever. Mem0 extracts fine-grained memory facts from the ingested text and retrieves over those facts; at query time, it returns the top-10 fact groups, and for node-level coverage metrics, these fact groups are aligned back to the source state-version nodes using the grouped source-node identifiers provided by Mem0. Graphiti builds a temporal knowledge graph over the ingested memories and applies a cross-encoder reranker to the retrieved candidates. Oracle Gold Context does not perform retrieval; instead, it directly provides the answer model with the annotated supporting memory units required by each question. We include Oracle Gold Context as an ideal-retrieval upper bound: it measures how much accuracy headroom remains once the correct state evidence is fully available to the answer model.

All baselines use the same normalized memory units and the same answer-generation model. In the final evaluation, answer generation and LLM-based judging both use deepseek-v4-pro with temperature 0.0 and thinking disabled. Dense retrieval uses BGE-M3 embeddings. Mem0 and Graphiti use deepseek-v4-flash as their internal memory-processing model, and Graphiti additionally uses a BGE Reranker v2 M3 model. The answer model is therefore

controlled across baselines, so the benchmark targets differences in memory retrieval rather than differences in the underlying LLM.

4.2 Overall Results

Table 5 reports the main results. Oracle Gold Context is treated as ideal retrieval, so its Cov and CSR are 1.0 by construction because the annotated supporting states are directly provided to the answer model. Its remaining errors appear only in ACC.

Among the automatic systems, FAISS is the strongest overall baseline. It achieves the best non-oracle ACC, Cov, and CSR on most task and retrieval-top-k settings, and it is the only automatic system that remains relatively stable as the task moves from single-state lookup to cross-version comparison and temporal ordering. This makes dense node-level retrieval a strong reference point for the benchmark. However, FAISS still degrades substantially on the harder tasks, which shows that semantic similarity alone is not sufficient once multiple versions are topically close but temporally different. One notable exception is cross-version comparison at $k=2$, where Graphiti (ACC 0.5467) slightly outperforms FAISS (ACC 0.5033), suggesting that graph-based structure and reranking can help when the retrieval budget is tight.

The other baselines reveal complementary weaknesses. BM25 remains serviceable on single-state lookup but degrades sharply on multi-state tasks, especially when the correct versions share substantial vocabulary with nearby distractor versions. Mem0 lags behind FAISS and Graphiti on all three task families, with the largest deficits on cross-version comparison and temporal ordering. This pattern suggests a granularity mismatch: retrieving fine-grained memory facts is not the same as recovering complete version states, because the benchmark’s gold support is defined over coherent version-level nodes rather than over isolated facts. Graphiti is stronger than BM25 and Mem0 on the harder tasks and is competitive with FAISS on cross-version comparison at low k , but it still trails FAISS overall, suggesting that structured memory organization and reranking do not by themselves solve version disambiguation at larger retrieval budgets.

4.3 Results by Task Type and Top-k

Figure 3 reports the results by task family and retrieval top-k.

The first trend concerns the effect of larger retrieval top-k. Coverage usually increases with k , especially on cross-version comparison and temporal ordering, but answer accuracy does not rise at the same rate. This gap is clearest for Mem0 and Graphiti. On temporal ordering, Mem0’s COV rises from 0.5556 at $k=5$ to 0.6356 at $k=10$, while ACC rises only from 0.1867 to 0.2600. Graphiti shows a similar pattern: its temporal-ordering COV increases from 0.6503 to 0.7136, yet ACC slightly decreases from 0.4833 at $k=8$ to 0.4767 at $k=10$. Even Oracle Gold Context shows a mild version of this effect on cross-version comparison. Larger retrieval top-k therefore helps recover more gold evidence, but it can also introduce more temporally adjacent distractors and increase the burden of evidence integration.

The second trend is a consistent difficulty ordering across baselines: single-state lookup is easiest, cross-version comparison is harder, and temporal ordering is hardest. As the number of required supporting states grows from one to two and then to three-to-five,

Table 4: Systems and configurations used in the formal benchmark. Oracle Gold Context bypasses retrieval and directly supplies the annotated supporting memory units to the answer model.

System	Retrieval unit	Internal memory model	Embedding / reranking	Answer model
BM25	memory unit text	none	lexical matching	DeepSeek-V4-Pro
FAISS	memory unit text	none	BGE-M3 embeddings	DeepSeek-V4-Pro
Mem0	memory facts	DeepSeek-V4-Flash	BGE-M3 embeddings	DeepSeek-V4-Pro
Graphiti	graph episodes / nodes	DeepSeek-V4-Flash	BGE-M3 embeddings + BGE Reranker v2 M3	DeepSeek-V4-Pro
Oracle Gold Context	annotated supporting memory units	none	none	DeepSeek-V4-Pro

Table 5: Main results on TSSMRBench. Each cell reports ACC / Cov / CSR, where ACC is answer accuracy, Cov is the average fraction of gold supporting states recovered in the retrieved set, and CSR is the fraction of questions for which the full gold support set is recovered. Oracle Gold Context is shown as an upper bound and is excluded from strongest-baseline marking. Bold marks the best automatic baseline and underline marks the second-best automatic baseline for each metric under the same task and retrieval top-k setting.

Task	System	Top-k 1	Top-k 2	Top-k 3
Single-state lookup (top-k cells: $k = 1, k = 2, k = 3$; values are ACC / Cov / CSR)				
Single	BM25	0.8533 / 0.8333 / 0.8333	0.8633 / 0.8733 / 0.8733	0.8800 / 0.8900 / 0.8900
	FAISS	0.8933 / 0.8667 / 0.8667	0.9133 / 0.9200 / 0.9200	0.9433 / 0.9500 / 0.9500
	Mem0	0.6433 / 0.6733 / 0.6733	0.6533 / 0.7033 / 0.7033	0.6567 / 0.7100 / 0.7100
	Graphiti	0.6800 / 0.6467 / 0.6467	0.7300 / 0.7167 / 0.7167	0.7267 / 0.7267 / 0.7267
	Oracle Gold Context	0.9967 / 1.0000 / 1.0000	1.0000 / 1.0000 / 1.0000	1.0000 / 1.0000 / 1.0000
Cross-version comparison (top-k cells: $k = 2, k = 5, k = 8$; values are ACC / Cov / CSR)				
Cross	BM25	0.2100 / 0.2767 / 0.1233	0.3200 / 0.3933 / 0.2567	0.3933 / 0.4483 / 0.3033
	FAISS	0.5033 / 0.6033 / 0.3433	0.7467 / 0.8067 / 0.6467	0.8400 / 0.8917 / 0.7933
	Mem0	0.2700 / 0.4400 / 0.1900	0.4067 / 0.5917 / 0.3833	0.4467 / 0.6400 / 0.4567
	Graphiti	0.5467 / 0.6200 / 0.3933	<u>0.6567 / 0.7150 / 0.5100</u>	<u>0.6700 / 0.7417 / 0.5467</u>
	Oracle Gold Context	0.9900 / 1.0000 / 1.0000	0.9833 / 1.0000 / 1.0000	0.9800 / 1.0000 / 1.0000
Temporal ordering (top-k cells: $k = 5, k = 8, k = 10$; values are ACC / Cov / CSR)				
Temporal	BM25	0.2767 / 0.5406 / 0.2033	0.3267 / 0.5803 / 0.2600	0.3200 / 0.6094 / 0.2867
	FAISS	0.4633 / 0.7036 / 0.3033	0.5633 / 0.8200 / 0.5200	0.6300 / 0.8606 / 0.6233
	Mem0	0.1867 / 0.5556 / 0.1733	0.2500 / 0.6136 / 0.2733	0.2600 / 0.6356 / 0.3067
	Graphiti	0.3767 / 0.6503 / 0.2733	<u>0.4833 / 0.6975 / 0.3533</u>	<u>0.4767 / 0.7136 / 0.3767</u>
	Oracle Gold Context	0.9067 / 1.0000 / 1.0000	0.9100 / 1.0000 / 1.0000	0.9100 / 1.0000 / 1.0000

the evaluated memory systems and retrieval baselines must do more than retrieve a relevant topic; they must identify a temporally compatible subset of a version trajectory. This pattern also helps explain the relative behavior of the baselines. FAISS remains strongest because dense node retrieval is effective at locating semantically close versions, but its performance still drops once several nearby versions are all topically relevant. BM25 declines more sharply because lexical overlap is an even coarser proxy for version identity. Mem0 gains coverage as top-k grows, yet much of the added evidence arrives as fragmented facts rather than coherent state versions, which weakens the conversion from retrieval support to correct answers. Graphiti benefits from structure and reranking on the harder tasks, but its gains still plateau, suggesting that retrieving the right memory region is not the same as preserving the fine-grained version boundaries required by these questions.

To quantify the task effect beyond the descriptive plots, we fit a mixed-effects logistic regression on answer correctness using the automatic baselines at the main task-specific retrieval top-k settings. The model uses fixed effects for task family and system, with a random intercept for question. Table 6 shows that, relative to

Table 6: Mixed-effects logistic regression on answer correctness at the main task-specific retrieval top-k settings. The model uses fixed effects for task family and system, with a random intercept for question. Odds ratios are reported relative to single-state lookup.

Contrast	Odds ratio	95% interval
Cross vs Single	0.26	[0.23, 0.30]
Temporal vs Single	0.11	[0.09, 0.12]

single-state lookup, the odds of a correct answer drop to 0.26× for cross-version comparison and to 0.11× for temporal ordering. This provides model-based support for the same conclusion suggested by the curves: retrieving the stage-correct state becomes substantially harder once the question depends on multiple temporally related versions.

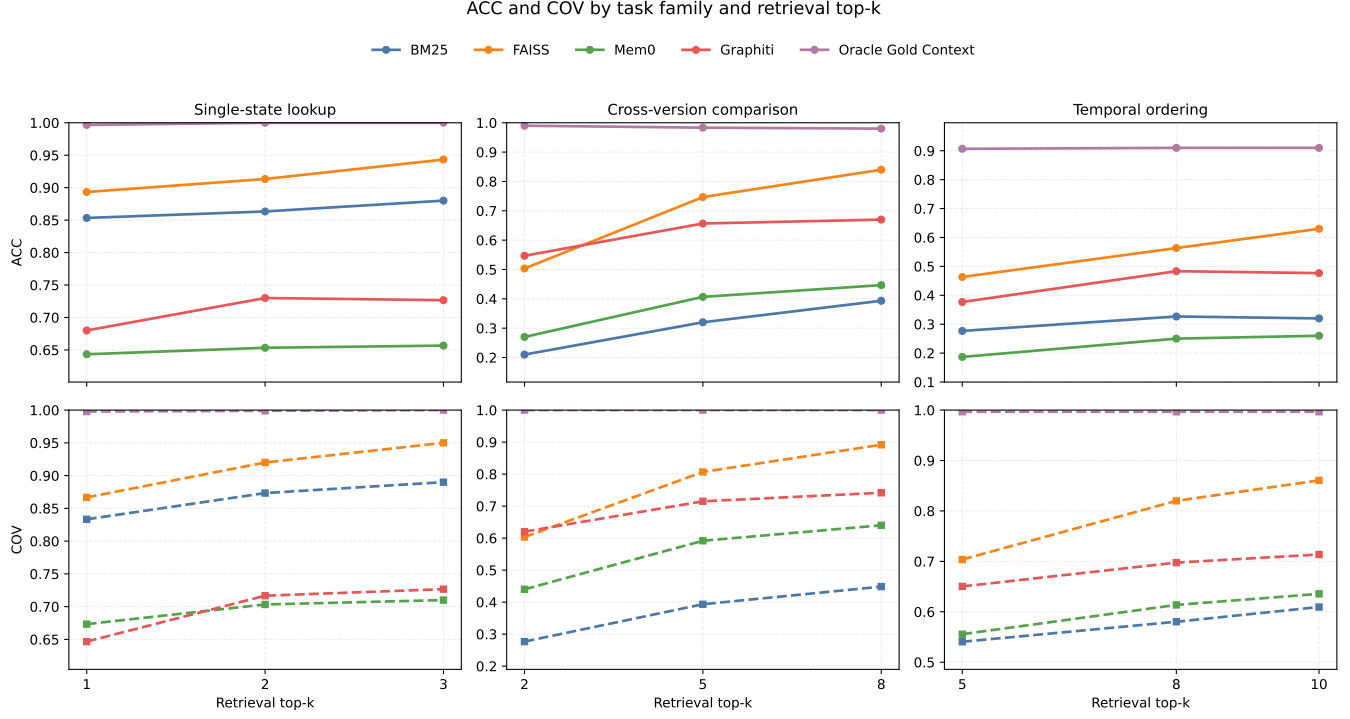


Figure 3: ACC and Cov by task family and retrieval top-k. The top row reports ACC and the bottom row reports Cov; the three columns correspond to single-state lookup, cross-version comparison, and temporal ordering.

4.4 Decoupling Retrieval from Reasoning

To better diagnose failure modes, we decouple retrieval success from final answer correctness. For each question, we categorize the retrieval outcome into three buckets: *all-recalled*, where all annotated supporting states are retrieved; *partial-recalled*, where only part of the support set is retrieved; and *zero-recalled*, where none of the supporting states are retrieved. We then split each bucket by whether the final answer is correct or incorrect.

This analysis is important because overall answer accuracy alone cannot tell us whether a memory system fails by missing relevant memories entirely, by retrieving incomplete support, or by retrieving related but temporally incompatible evidence. In particular, the *all-recalled but incorrect* bucket is highly diagnostic: it indicates that the memory system has already retrieved the necessary state evidence but the final answer is still not resolved correctly. Conversely, the *partial-recalled but correct* and *zero-recalled but correct* buckets indicate cases where the answer may be guessed from partial cues or option bias rather than supported by complete evidence.

Figure 4 shows that the baselines fail for different reasons, and these differences are informative from a systems perspective. FAISS has the strongest retrieval profile: 73.44% of all questions fall into the all-recalled and correct bucket, while only 2.00% fall into zero-recalled and incorrect. Its remaining errors are therefore less about complete retrieval failure and more about selecting among semantically adjacent versions after the right region of memory has already been retrieved. This is consistent with the behavior of dense semantic search: it is good at getting close to the target state, but it has no

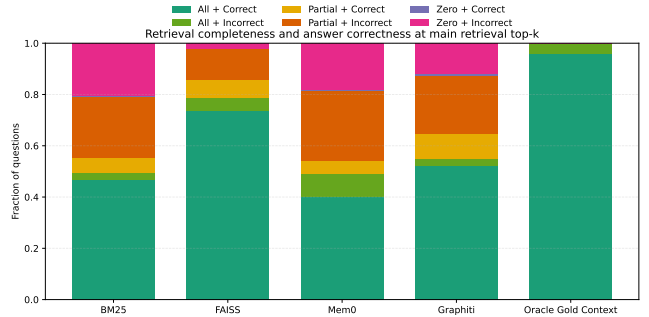


Figure 4: Retrieval-reasoning decoupling at the main task-specific retrieval top-k settings. Each bar partitions questions by retrieval completeness and final answer correctness.

explicit mechanism for preserving version lineage or distinguishing neighboring historical states.

Mem0 presents a different pattern once its fact-level retrieval is aligned back to source state nodes through grouped source-node identifiers. Under this corrected accounting, only 0.22% of all questions fall into zero-recalled and correct, confirming that Mem0 rarely answers correctly without relevant evidence. Its dominant error buckets are instead partial-recalled and incorrect (27.44%) and all-recalled but incorrect (8.89%). Because Mem0 retrieves individual memory facts rather than complete version states, it frequently

Table 7: Computational cost profile. Latency is mean retrieval-only latency in milliseconds, excluding answer generation. Retrieved context length is the mean number of context tokens passed to the answer model at the primary retrieval top-k setting of each task family.

System	Latency	Single	Cross	Temporal
BM25	1110.63	181.41	402.16	438.14
FAISS	195.12	270.17	611.98	856.29
Mem0	417.71	90.72	174.46	214.38
Graphiti	34425.76	224.58	539.11	836.39
Oracle Gold Context	0.00	87.64	169.43	309.78

recovers only part of the required support. In multi-state questions, this partial recovery weakens support completeness and makes it harder for the answer model to reconstruct a temporally compatible trajectory.

Graphiti suggests a third failure mode. It is stronger than Mem0 on cross-version comparison and temporal ordering, which shows the benefit of structure and reranking, but it still has a large partial-recalled and incorrect segment (22.78%). Graphiti’s graph construction process builds temporal edges and summarized entity nodes over the ingested memories, which can help organize long histories but may also reduce the distinctiveness of individual version states. For TSSMRBench, this is harmful because the correct answer often depends on preserving distinct historical states rather than collapsing them into one generalized representation.

These findings point to a broader design implication. A memory system for evolving information cannot rely only on semantic relevance, nor can it simply overwrite old memories with newer ones. It must preserve version-level history, retrieve coherent state units rather than only fragments, and support multi-state evidence assembly without losing temporal compatibility.

4.5 Computational Efficiency

Table 7 shows that effectiveness must be read together with computational cost. FAISS is the strongest efficiency–effectiveness trade-off among the automatic baselines: it is the fastest retriever (195.12 ms on average) and also the strongest automatic system in accuracy, coverage, and complete-support rate. BM25 is slower than FAISS despite weaker overall effectiveness, which makes it a useful lexical reference point but not a strong efficiency frontier.

Mem0 occupies a different point in the trade-off space. Its latency remains moderate, and its retrieved contexts are the shortest among the automatic systems because it operates on compact memory facts. However, this apparent efficiency advantage comes with a clear loss in state-level effectiveness. The system retrieves smaller memory units quickly, but those units are often too fragmented to support reliable version-level reasoning. In other words, cheaper context is not necessarily better context.

Graphiti exposes the opposite extreme. It retrieves contexts of similar scale to FAISS on the harder tasks, but its retrieval latency is much higher (34,425.76 ms on average). This cost likely reflects the additional graph traversal, candidate expansion, and reranking needed to retrieve structured memories from a global mixed pool. For TSSMRBench, the practical question is whether this extra structure leads to clearly better version-sensitive retrieval at an

acceptable cost. Our results show that this is not yet guaranteed. Future memory systems therefore need to satisfy both requirements at once: they should preserve version history faithfully, and they should still retrieve the needed states efficiently from large mixed memory pools.

5 Conclusion

We presented TSSMRBench, a benchmark for temporal semantic state memory retrieval over evolving memories. TSSMRBench evaluates whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory, rather than merely returning a relevant or recent memory.

To study this ability, we constructed a benchmark dataset with 300 scenarios, 9,000 memory nodes, and 900 questions, organized into three task families: single-state lookup, cross-version comparison, and temporal ordering. This design lets us test not only whether a memory system retrieves relevant memory, but also whether it retrieves the temporally correct state evidence needed by questions of increasing multi-state complexity.

Our experiments show that current memory systems are substantially stronger on single-state lookup than on cross-version comparison and temporal ordering, and that many failures arise even when semantically related evidence is present in memory. Sparse and dense retrieval backends often retrieve topically relevant versions but do not explicitly preserve version identity; fact-based memory systems recover individual facts rather than complete version states, leading to partial support; and structured memory systems can reduce the distinctiveness of individual historical states through graph construction and summarization. These results highlight the challenge of temporal semantic state retrieval for current memory systems and clarify an important future design direction: memory systems should preserve version history, retrieve coherent state-level evidence, support stage-aware selection among semantically similar memories, and do so efficiently over large shared memory pools.

Artifacts

The source code, formal benchmark dataset, and evaluation results are publicly available in the project repository.

<https://github.com/ngyygm/llm-TSSMRBench>

References

- [1] Wenhu Chen, Xinyi Wang, and William Yang Wang. 2021. A Dataset for Answering Time-Sensitive Questions. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. Curran Associates, Inc., Virtual. <https://arxiv.org/abs/2108.06314>
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413 [cs.CL] doi:10.48550/arXiv.2504.19413
- [3] Pengfei Du. 2026. Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers. arXiv:2603.07670 [cs.AI] doi:10.48550/arXiv.2603.07670
- [4] Bahare Fatemi, Mehran Kazemi, Anton Tsitsulin, Karishma Malkan, Jinyeong Yim, John Palowitch, Sungyong Seo, Jonathan Halcrow, and Bryan Perozzi. 2024. Test of Time: A Benchmark for Evaluating LLMs on Temporal Reasoning. arXiv:2406.09170 [cs.CL] <https://arxiv.org/abs/2406.09170>
- [5] Yunfan Gao, Yun Xiong, Xinyu Gao, Kang Jia, Jinliu Pan, Yuxian Bi, Yutong Dai, Jiawei Sun, Meng Wang, and Haoften Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs.CL] doi:10.48550/arXiv.2312.10997
- [6] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. arXiv:2402.01680 [cs.CL] doi:10.48550/arXiv.2402.01680
- [7] Yuanzhe Hu, Yu Wang, and Julian McAuley. 2025. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. arXiv:2507.05257 [cs.CL] doi:10.48550/arXiv.2507.05257
- [8] Christian S. Jensen and Richard T. Snodgrass. 1999. Temporal Data Management. *IEEE Transactions on Knowledge and Data Engineering* 11, 1 (1999), 36–44.
- [9] Tomáš Kočíský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, and Gábor Melis. 2018. The NarrativeQA Reading Comprehension Challenge. *Transactions of the Association for Computational Linguistics* 6 (2018), 317–328. doi:10.1162/tacl_a_00023
- [10] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. doi:10.1162/tacl_a_00276
- [11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Virtual, 9459–9474.
- [12] Sigang Luo, Yanan Liu, Dongying Lin, Yingying Zhai, Bin Wang, Xiaochun Yang, and Junpeng Liu. 2025. ETRQA: A Comprehensive Benchmark for Evaluating Event Temporal Reasoning Abilities of Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Vienna, Austria, 23321–23339. doi:10.18653/v1/2025.findings-acl.1198
- [13] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 13851–13870. doi:10.18653/v1/2024.acl-long.747
- [14] Qiang Ning, Hao Wu, Rujun Han, Nanyun Peng, Matt Gardner, and Dan Roth. 2020. TORQUE: A Reading Comprehension Dataset of Temporal Ordering Questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online, 1158–1172. doi:10.18653/v1/2020.emnlp-main.88
- [15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] doi:10.48550/arXiv.2310.08560
- [16] Lianhui Qin, Aditya Gupta, Shyam Upadhyay, Luheng He, Yejin Choi, and Manaal Faruqi. 2021. TIMEDIAL: Temporal Commonsense Reasoning in Dialog. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, Online, 7066–7076. doi:10.18653/v1/2021.acl-long.549
- [17] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956 [cs.CL] doi:10.48550/arXiv.2501.13956
- [18] Richard T. Snodgrass. 1995. *The SQL2 Temporal Query Language*. Kluwer Academic Publishers, Boston, MA.
- [19] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554. doi:10.1162/tacl_a_00475
- [20] Shaohang Wei, Wei Li, Feifan Song, Wen Luo, Tianyi Zhuang, Haochen Tan, Zhijiang Guo, and Houfeng Wang. 2025. TimE: A Multi-level Benchmark for Temporal Reasoning of LLMs in Real-World Scenarios. arXiv:2505.12891 [cs.CL] doi:10.48550/arXiv.2505.12891
- [21] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813 [cs.CL] doi:10.48550/arXiv.2410.10813
- [22] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110 [cs.AI] doi:10.48550/arXiv.2502.12110
- [23] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 2369–2380. doi:10.18653/v1/D18-1259
- [24] Tingyi Zhang, Jiaan Wang, Zhixu Li, Jianfeng Qu, An Liu, Zhigang Chen, and Hongping Zhi. 2024. MusTQ: A Temporal Knowledge Graph Question Answering Dataset for Multi-Step Temporal Reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 11688–11699. doi:10.18653/v1/2024.findings-acl.696

A Appendix

A.1 Exact Prompt Excerpts

We include exact prompt excerpts here for reproducibility. Listings 1–3 show the final construction and answer-generation prompts used in the released benchmark.

Listing 1: Exact excerpt from the final release-window construction prompt.

```
Build one unified repository release-evolution prototype from the provided official GitHub release-note nodes.

Requirements:
- Use only the provided release-note nodes.
- Produce one single JSON object matching the output schema.
- Only generate chunk-level memory_unit_text values keyed by memory_node_id.
- For every chunk, write memory_unit_text in English.
- Every memory_unit_text must explicitly name both the repository and the current release version or tag.
- Every memory_unit_text must describe only the current release note itself.
- Do not compare to earlier or later releases inside memory_unit_text.
- Do not mention benchmark setup, memory pools, retrieval, neighboring releases, hidden context, or any chain-construction process.
...
```

Listing 2: Exact prompt excerpt defining the final output schema and QA rules.

```
"questions": [
  {
    "question_id": "string",
    "task_type": "single_state_lookup / cross_version_comparison / temporal_version_ordering",
    "answer_format": "multiple_choice",
    "query_text": "question text that explicitly names the repository",
    "options": [
      {"option_id": "A", "text": "option text"},
      {"option_id": "B", "text": "option text"},
      {"option_id": "C", "text": "option text"},
      {"option_id": "D", "text": "option text"}
    ],
    "correct_option_id": "A",
    "source_chunk_ids": ["memory_node_id_1"],
    "answer_support": [{"memory_node_id": "memory_node_id_1", "answer_text": "support text"}]
  }
]

- Generate exactly 3 multiple-choice questions, one per task type.
- Every query_text must explicitly name the repository.
- Use source_chunk_ids as the question-level gold evidence field.
- For cross_version_comparison, compare two releases using concrete release-content differences.
- For temporal_version_ordering, use 3 to 5 release-content states.
- The answer options for temporal_version_ordering should only be orderings of those indices.
- The question should still require the system to retrieve the corresponding chunk-level memory_unit_text content.
...
```

Listing 3: Exact excerpt from benchmark/src/evaluation/answer_generator.py.

```
MC_SYSTEM_PROMPT = """You are a strict evaluation assistant for a memory benchmark.

You must answer using only the retrieved memory.
You must not use outside knowledge, background knowledge, guessing, or prior world knowledge.
Your job is to choose the best-supported answer from the retrieved memory.

Output rule:
- Return exactly one uppercase option letter: A, B, C, D, or E.
- Do not output any explanation, punctuation, JSON, markdown, or extra text.
- If the retrieved memory contains relevant evidence for one option, choose that option actively.
- Return E only when the retrieved memory is completely irrelevant or contains no answer-bearing evidence for any option."""

MC_USER_PROMPT = """You may answer only from the retrieved memory below.
If the retrieved memory contains relevant evidence for one option, choose the best-supported option.
Return E only if the retrieved memory is completely irrelevant or contains no answer-bearing evidence.
...
"""
```

A.2 Exact Dataset Excerpts

Listing 4 shows a real excerpt from the unified formal dataset file for the Raphire/Win11Debloat repository window used in Figure 1.

Listing 4: Exact excerpt from official_300_merged.json for Raphire/Win11Debloat.

```
{
  "repo": "Raphire/Win11Debloat",
  "window_title": "Raphire/Win11Debloat recent release evolution",
  "chunk_example": {
    "memory_node_id": "Raphire__Win11Debloat__release__2025_06_11",
    "repo": "Raphire/Win11Debloat",
  }
}
```

```

    "tag_name": "2025.06.11",
    "title": "Release 2025.06.11",
    "time_hint": "2025-06-10",
    "published_at": "2025-06-10T22:10:42Z",
    "raw_text": "Repository: Raphire/Win11Debloat ...",
    "memory_unit_text": "In the Raphire/Win11Debloat release 2025.06.11, this minor release fixes an issue with clearing the start menu pinned apps layout for all users
    .",
  },
  "question_example": {
    "question_id": "q3_temporal_version_ordering",
    "task_type": "temporal_version_ordering",
    "answer_format": "multiple_choice",
    "query_text": "In the Raphire/Win11Debloat release window, order the following release-content states from earliest to latest: (1) a release that adds a full
    graphical user interface (GUI), (2) a release that adds an option to disable AI features in Paint & Notepad, (3) a release that adds System Registry backup &
    restore, (4) a release that adds an option to disable BitLocker automatic device encryption.",
    "options": [
      { "option_id": "A", "text": "2 -> 1 -> 4 -> 3" },
      { "option_id": "B", "text": "2 -> 4 -> 1 -> 3" },
      { "option_id": "C", "text": "1 -> 2 -> 3 -> 4" },
      { "option_id": "D", "text": "4 -> 2 -> 1 -> 3" }
    ],
    "correct_option_id": "A",
    "source_chunk_ids": [
      "Raphire__Win11Debloat__release__2025_08_01",
      "Raphire__Win11Debloat__release__2026_02_01",
      "Raphire__Win11Debloat__release__2026_02_04",
      "Raphire__Win11Debloat__release__2026_05_10"
    ],
    "answer_support": [
      { "memory_node_id": "Raphire__Win11Debloat__release__2025_08_01", "answer_text": "an option to disable AI features in Paint & Notepad is introduced" },
      ...
    ]
  }
}
}

```

A.3 Exact Retrieval Result Excerpts

Listings 5 and 6 show exact successful and failed records from the formal Mem0 evaluation on the same repository window.

Listing 5: Exact successful Mem0 result excerpt from mem0.questions.jsonl.

```

{
  "question_id": "Raphire__Win11Debloat_release_window__q1_single_state_lookup",
  "query_text": "In the Raphire/Win11Debloat releases, which release adds an option to disable AI features in Paint & Notepad?",
  "retrieved_facts": [
    "- In the Raphire/Win11Debloat release 2025.08.01, an option to disable AI features in Paint & Notepad is introduced.",
    "- In the Raphire/Win11Debloat release 2025.08.16, a missing key for Disable Paint AI Sysprep/Undo files is fixed. ...",
    "- In the Raphire/Win11Debloat release 2025.06.12, new options are added to disable animations and visual effects. ..."
  ],
  "matched_gold_node_ids": [
    "Raphire__Win11Debloat__release__2025_08_01"
  ],
  "support_coverage": 1.0,
  "generated_answer": "B",
  "is_correct": true
}

```

Listing 6: Exact failed Mem0 temporal-ordering result excerpt from mem0.questions.jsonl.

```

{
  "question_id": "Raphire__Win11Debloat_release_window__q3_temporal_version_ordering",
  "query_text": "In the Raphire/Win11Debloat release window, order the following release-content states from earliest to latest: ...",
  "retrieved_facts": [
    "- In the Raphire/Win11Debloat release 2025.08.01, an option to disable AI features in Paint & Notepad is introduced.",
    "- In the Raphire/Win11Debloat release 2026.02.01-R2, a full graphical user interface (GUI) is introduced.",
    "- In the Raphire/Win11Debloat release 2026.04.26, the app & tweak selection UI is improved ...",
    "- In the Raphire/Win11Debloat release 2026.02.12, the DisableEdgeAds feature is improved ..."
  ],
  "matched_gold_node_ids": [
    "Raphire__Win11Debloat__release__2025_08_01"
  ],
  "support_coverage": 0.25,
  "generated_answer": "E",
  "is_correct": false
}

```

In the formal release-only experiments, the evaluated memory systems ingest only `memory_unit_text`. Raw release-note text is preserved for auditing but is not written into the retrieval pool. The released result files also retain the original prompt payload, retrieved-memory text, matched supporting node identifiers, and per- k sliced evaluation outputs so that every reported score can be traced back to concrete evidence.